**Społeczna Akademia Nauk**

Instytut Technologii Informatycznych

Kierunek studiów:

**INFORMATYKA**

Przedmiot:

**Systemy Szkieletowe**

# System do zarządzania sylabusami uczelni wyższej

**Kacper Juśkiewicz**
Nr albumu: 114725
Grupa: 4

Prowadzący:
**mgr inż. Krystian Gumiński**

Łódź 2026

# Spis treści

**1 Wstęp** **2**

**2 Cel projektu** **3**
2.1 Problem . . . 3
2.2 Cele . . . 3
2.3 Wymagania funkcjonalne . . . 3
2.4 Wymagania niefunkcjonalne . . . 4

**3 Przegląd istniejących rozwiązań** **5**
3.1 USOS . . . 5
3.2 Akademus . . . 5
3.3 Uczelnia.XP . . . 5
3.4 Porównanie . . . 6

**4 Opis architektury** **7**
4.1 Architektura ogólna . . . 7
4.2 Warstwy backendu . . . 7
4.3 Komunikacja między komponentami . . . 8
4.4 Zastosowane technologie . . . 10

**5 Opis implementacji** **11**
5.1 Frontend . . . 11
5.2 Backend . . . 15
5.2.1 Uwierzytelnianie i autoryzacja . . . 15
5.2.2 Research: logowanie kontem uczelnianym . . . 16
5.2.3 Wersjonowanie i historia zmian . . . 16
5.2.4 Walidacja danych . . . 17
5.2.5 Wyszukiwanie i siatka przedmiotów . . . 17
5.2.6 Eksport do PDF i DOCX . . . 18
5.3 Baza danych . . . 18
5.4 Docker . . . 19
5.5 Testy . . . 19
5.6 Monitoring . . . 20

**6 Podsumowanie i wnioski** **22**
6.1 Napotkane wyzwania . . . 22
6.2 Możliwości dalszego rozwoju . . . 22

**Bibliografia** **26**

# 1 Wstęp

Każdy przedmiot na studiach ma swój sylabus. To w nim prowadzący zapisuje, czego student ma się nauczyć, ile jest godzin wykładów i laboratoriów, za ile przedmiot jest punktów ECTS, jak wygląda zaliczenie i z jakich książek warto korzystać. Uczelnia musi te informacje udostępniać, co wynika z ustawy Prawo o szkolnictwie wyższym i nauce [1]. Zasady liczenia punktów ECTS opisuje z kolei europejski system ECTS [2].

Mimo to sylabusy bardzo często powstają w Wordzie i krążą mailem między prowadzącym a dziekanatem. Szybko robi się z tego kilka wersji tego samego pliku i nie wiadomo, która jest aktualna. Nie widać też, kto i kiedy coś w nim zmienił. Jeszcze trudniej jest na uczelni, która ma kilka ośrodków w różnych miastach. Ten sam przedmiot ma wtedy kilka sylabusów, które z roku na rok coraz bardziej się od siebie różnią. Sprawdzenie, czy liczba godzin pasuje do punktów ECTS, zwykle oznacza liczenie na kartce.

W ramach projektu napisałem aplikację **SyllabusApp**, czyli webowy system do zarządzania sylabusami. Sylabusy wprowadza się w niej przez formularze. Każda zmiana zapisuje się jako nowa wersja, więc historia dokumentu jest zawsze dostępna. Aplikacja pozwala też wyszukiwać i filtrować sylabusy, pokazuje siatkę przedmiotów dla kierunku, pilnuje poprawności liczby godzin i eksportuje sylabus do PDF albo DOCX.

Całość napisałem na platformie .NET 10. Backend to ASP.NET Core Web API [3], frontend powstał w Blazor WebAssembly [4], a dane trzyma baza PostgreSQL [5], z którą aplikacja łączy się przez Entity Framework Core [6]. Wszystko uruchamia się w kontenerach Docker [7]. Projekt ma też testy jednostkowe i prosty monitoring stanu aplikacji.

# 2 Cel projektu

## 2.1 Problem

Najważniejszy problem to brak jednego miejsca, w którym są aktualne sylabusy. Kiedy dokumenty leżą w różnych plikach i skrzynkach mailowych, każdy na tym traci. Prowadzący co roku poprawia te same treści od nowa. Dziekanat musi ręcznie sprawdzać, czy sylabusy zgadzają się z programem studiów. Student nie ma pewności, czy czyta aktualną wersję.

## 2.2 Cele

Chciałem zbudować aplikację webową, która:

- trzyma sylabusy w bazie danych i wiąże je z ośrodkiem, wydziałem, kierunkiem, przedmiotem i rokiem akademickim,
- ma formularze na wszystkie elementy sylabusa,
- przy każdej zmianie sama zapisuje nową wersję, żeby dało się odtworzyć historię,
- sprawdza poprawność danych, przede wszystkim to, czy godziny zgadzają się z ECTS,
- pozwala szybko znaleźć sylabus po kilku kryteriach,
- pokazuje siatkę przedmiotów kierunku z podziałem na semestry,
- eksportuje sylabus do PDF i DOCX,
- wpuszcza tylko zalogowanych użytkowników i rozróżnia ich role.

## 2.3 Wymagania funkcjonalne

W tabeli 1 zebrałem wymagania postawione przed projektem i to, jak je zrealizowałem.

Tabela 1: Wymagania funkcjonalne i sposób ich realizacji

| **Wymaganie** | **Realizacja w SyllabusApp** |
|---|---|
| Logowanie przy użyciu konta uczelnianego | Research dostępnych rozwiązań (OpenID Connect z Microsoft Entra ID, USOS API), opisany w podrozdziale 5.2.2. Logowanie oparte na JWT pozwala później dodać zewnętrznego dostawcę bez przebudowy reszty systemu |
| Rejestracja i logowanie loginem i hasłem | ASP.NET Core Identity (hasła zapisywane jako hash), token JWT, endpointy `/auth/register`, `/auth/login`, `/auth/me` |
| Formularze sylabusa (opis, cele, efekty, wymagania, literatura, ocenianie, godziny) | Formularz sylabusa oraz sekcje efektów uczenia się, literatury i nakładu pracy na stronie szczegółów |
| Wersjonowanie sylabusów | Każde utworzenie i edycja zapisuje nową wersję (tabela `SyllabusVersions`) z numerem, autorem, datą, opisem zmian i kopią danych w JSON |
| Historia zmian | Tabela „Historia zmian” w szczegółach sylabusa, endpoint `GET /syllabuses/{id}/versions` |
| Różne sylabusy dla ośrodków i roku wprowadzenia | Ośrodek → Wydział → Kierunek → Przedmiot → Sylabus, a w sylabusie pole z rokiem akademickim |
| Siatka przedmiotów | Widok „Siatka przedmiotów” z semestrami, godzinami wykładów, ćwiczeń i laboratoriów, punktami ECTS i sumami |
| Walidacja danych (liczba godzin) | Klasa `SyllabusRules`: zakres ECTS, godziny nie mniejsze od zera, godziny zajęć $\leq$ ECTS $\times$ 30, kontrola nakładu pracy 25–30 h na punkt |
| Wyszukiwanie i filtrowanie | Wyszukiwanie po tekście i filtry: ośrodek, kierunek, rok akademicki, status, semestr |
| Eksport do PDF i DOCX | Biblioteki QuestPDF i Open XML SDK, endpointy `/export/pdf` i `/export/docx` |

## 2.4 Wymagania niefunkcjonalne

Poza samymi funkcjami zależało mi na kilku ogólniejszych rzeczach. Do API można się dostać tylko z ważnym tokenem JWT, a uprawnienia zależą od roli użytkownika. Hasła nie są nigdzie zapisane wprost. Baza, API i frontend startują jednym poleceniem `docker compose up`, więc nie trzeba niczego instalować ręcznie.

Kod podzieliłem na warstwy według zasad czystej architektury [8]. Każda encja jest obsługiwana według tego samego schematu: DTO, serwis, kontroler, klient API i strony. Dzięki temu łatwo się w nim odnaleźć. Stan aplikacji można sprawdzić pod adresem `/health`, a logikę biznesową sprawdzają testy uruchamiane przez `dotnet test`. Interfejs jest po polsku, a komunikaty o błędach mówią wprost, co trzeba poprawić.

# 3 Przegląd istniejących rozwiązań

W Polsce sylabusy obsługuje kilka systemów. Zwykle są to moduły dużych systemów do zarządzania całą uczelnią.

## 3.1 USOS

USOS, czyli Uniwersytecki System Obsługi Studiów [9], rozwija Międzyuniwersyteckie Centrum Informatyzacji. Korzysta z niego kilkadziesiąt polskich uczelni. Opisy przedmiotów leżą w katalogu przedmiotów i są publikowane w serwisie USOSweb. System obsługuje cykle dydaktyczne, efekty uczenia się powiązane z Polską Ramą Kwalifikacji i rozbudowane programy studiów. Ma też publiczne API [10] z autoryzacją OAuth, przez które mogą się z nim łączyć inne aplikacje.

Zaletą USOS-a jest ogromny zakres funkcji i połączenie z rekrutacją, ocenami i planem zajęć. Wadą jest to, że wdrożenie i utrzymanie są skomplikowane, a edycja sylabusów nie należy do wygodnych. System jest też przeznaczony głównie dla uczelni publicznych z konsorcjum.

## 3.2 Akademus

Akademus [11] to komercyjny system do zarządzania uczelnią. Ma między innymi moduł programów studiów i sylabusów. Można w nim tworzyć karty przedmiotów, przypisywać efekty uczenia się, generować dokumenty i publikować je na stronie uczelni.

Plusem są gotowe szablony dokumentów i moduł sprawozdawczy. Minusem jest to, że system jest zamknięty i płatny, a sylabusy to tylko jego mały fragment, więc nie da się łatwo wdrożyć samego tego modułu.

## 3.3 Uczelnia.XP

Uczelnia.XP [12] firmy Asseco Data Systems to system obsługi toku studiów. W module planów i programów studiów przygotowuje się w nim sylabusy zgodne z PRK. Jest też obieg zatwierdzania dokumentów.

System dobrze współpracuje z dziekanatem i ma wsparcie producenta. Z drugiej strony licencja jest droga, interfejs dość ciężki, a uczelnia uzależnia się od jednego dostawcy.

## 3.4 Porównanie

Tabela 2: Porównanie istniejących rozwiązań z SyllabusApp

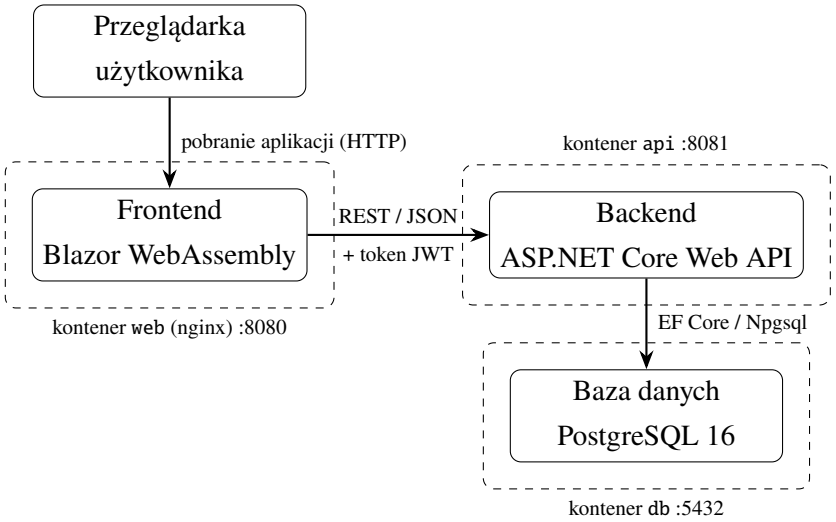
| **Cecha** | **USOS** | **Akademus** | **Uczelnia.XP** | **SyllabusApp** |
|---|---|---|---|---|
| Formularze sylabusa | tak | tak | tak | tak |
| Historia wersji sylabusa | częściowo | częściowo | tak | tak |
| Siatka przedmiotów | tak | tak | tak | tak |
| Walidacja godzin/ECTS | tak | tak | tak | tak |
| Eksport PDF | tak | tak | tak | tak |
| Eksport DOCX | nie | tak | tak | tak |
| Wiele ośrodków | tak | tak | tak | tak |
| Lekkie wdrożenie (Docker) | nie | nie | nie | tak |
| Otwarte REST API | tak | nie | nie | tak |
| Koszt licencji | konsorcjum | płatny | płatny | brak |

Wszystkie trzy systemy są duże i obsługują całą uczelnię, a sylabus jest w nich tylko jednym z wielu modułów. SyllabusApp robi wyłącznie sylabusy. Przez to jest dużo mniejszy i łatwiejszy do uruchomienia, a mimo to ma najważniejsze funkcje: wersjonowanie, siatkę przedmiotów, walidację i eksport.

# 4 Opis architektury

## 4.1 Architektura ogólna

Aplikacja działa w układzie klient–serwer i składa się z trzech części: frontendu działającego w przeglądarce, serwera REST API i bazy danych. Każda z nich ma swój kontener Docker (rys. 1).




Rysunek 1: Architektura systemu SyllabusApp

Frontend to zestaw statycznych plików, które serwuje nginx. Przeglądarka pobiera je i uruchamia kod .NET w WebAssembly. Od tego momentu aplikacja rozmawia z API, wysyłając zapytania HTTP z danymi w JSON. API sprawdza token użytkownika, wykonuje logikę i zapisuje dane w PostgreSQL.

## 4.2 Warstwy backendu

Backend podzieliłem na cztery projekty. Zależności idą tylko w jedną stronę, do środka, więc domena nie wie nic o bazie danych ani o API (tab. 3).

Tabela 3: Projekty (warstwy) rozwiązania

| **Projekt** | **Odpowiedzialność** | **Zależy od** |
|---|---|---|
| `SyllabusApp.Domain` | Encje (Sylabus, Przedmiot, Kierunek, . . . ), enumeracje, stałe ról | – |
| `SyllabusApp.Application` | DTO, mapowania, interfejsy serwisów, reguły walidacji `SyllabusRules` | Domain |
| `SyllabusApp.Infrastructure` | `DbContext`, migracje, implementacje serwisów, generowanie JWT, eksport PDF/DOCX | Application |
| `SyllabusApp.Api` | Kontrolery REST, konfiguracja uwierzytelniania, CORS, OpenAPI, health checks | Application, Infrastructure |
| `SyllabusApp.Web` | Frontend Blazor WebAssembly: strony, klienci API, obsługa logowania | Application |
| `SyllabusApp.Tests` | Testy jednostkowe (xUnit) | Application, Infrastructure |

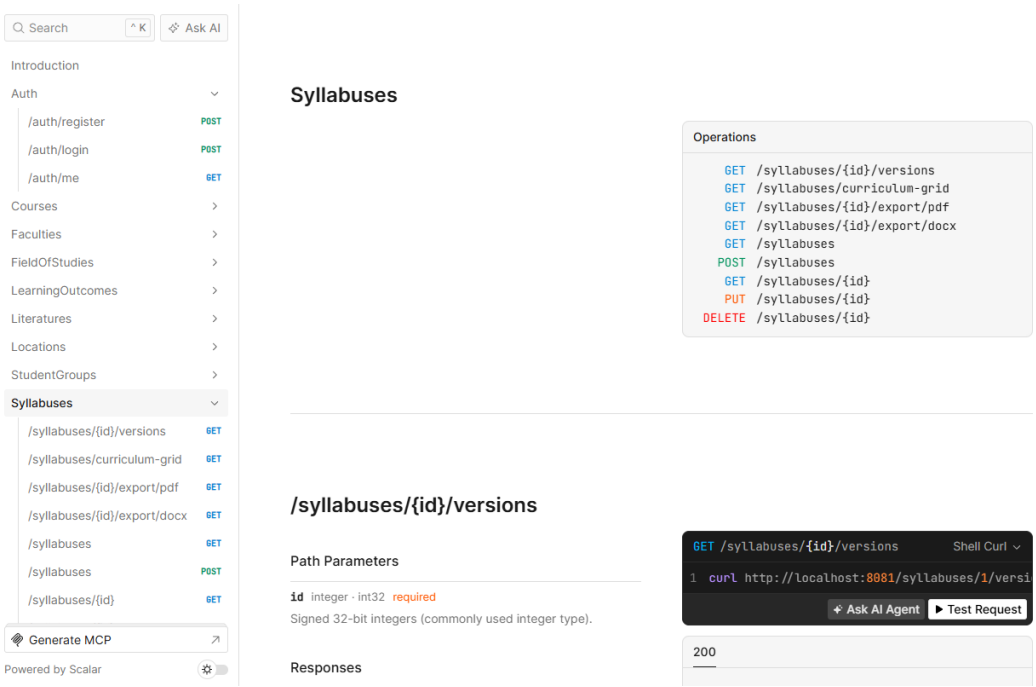
Jedną z ważniejszych decyzji było to, żeby projekt `Application` był wspólny dla backendu i frontendu. Te same klasy DTO, razem z atrybutami walidacji, są używane po obu stronach. Formularz sprawdza więc dane już w przeglądarce, a serwer sprawdza je jeszcze raz, przy czym reguły są zapisane tylko w jednym miejscu.

## 4.3 Komunikacja między komponentami

Frontend rozmawia z backendem tylko przez REST API. Oprócz logowania i rejestracji każde zapytanie ma nagłówek `Authorization: Bearer <token>`. Token JWT [13] wydaje endpoint `/auth/login`. Przeglądarka trzyma go w `localStorage`, a `AuthorizationMessageHandler` sam dokleja go do zapytań. Wartości enumeracji przesyłam jako tekst, bo łatwiej się wtedy czyta odpowiedzi API. Dokumentacja API generuje się automatycznie w formacie OpenAPI i można ją przeglądać w interfejsie Scalar [14] (rys. 2).

Tabela 4: Najważniejsze endpointy REST API

| Metoda | Ścieżka | Opis |
|---|---|---|
| POST | `/auth/register`, `/auth/login` | Rejestracja i logowanie, zwraca token JWT |
| GET | `/auth/me` | Dane zalogowanego użytkownika |
| GET | `/syllabuses?search=&locationId=...` | Lista sylabusów z wyszukiwaniem i filtrami |
| GET/POST | `/syllabuses` | Odczyt i tworzenie sylabusów |
| GET/PUT/DELETE | `/syllabuses/{id}` | Szczegóły, edycja (nowa wersja), usunięcie |
| GET | `/syllabuses/{id}/versions` | Historia wersji sylabusa |
| GET | `/syllabuses/{id}/export/pdf` | Eksport do PDF |
| GET | `/syllabuses/{id}/export/docx` | Eksport do DOCX |
| GET | `/syllabuses/curriculum-grid` | Siatka przedmiotów kierunku |
| CRUD | `/locations`, `/faculties`, ... | Słowniki: ośrodki, wydziały, kierunki, przedmioty, grupy |
| CRUD | `/learning-outcomes`, `/literatures`, `/workload-items` | Elementy sylabusa |
| GET | `/health` | Stan aplikacji i połączenia z bazą |




Rysunek 2: Interaktywna dokumentacja API w Scalarze

## 4.4 Zastosowane technologie

Tabela 5: Stos technologiczny

| **Obszar** | **Technologia** |
|---|---|
| Język i platforma | C# 14, .NET 10 |
| Backend | ASP.NET Core Web API, ASP.NET Core Identity, JWT Bearer |
| Frontend | Blazor WebAssembly, Tailwind CSS 4 [15] |
| Baza danych | PostgreSQL 16, Entity Framework Core 10, Npgsql [16] |
| Dokumentacja API | OpenAPI (Microsoft.AspNetCore.OpenApi), Scalar |
| Eksport dokumentów | QuestPDF [17], DocumentFormat.OpenXml [18] |
| Testy | xUnit [19], EF Core InMemory |
| Konteneryzacja | Docker, Docker Compose, nginx |
| Monitoring | ASP.NET Core Health Checks [20], healthchecki Docker |

# 5 Opis implementacji

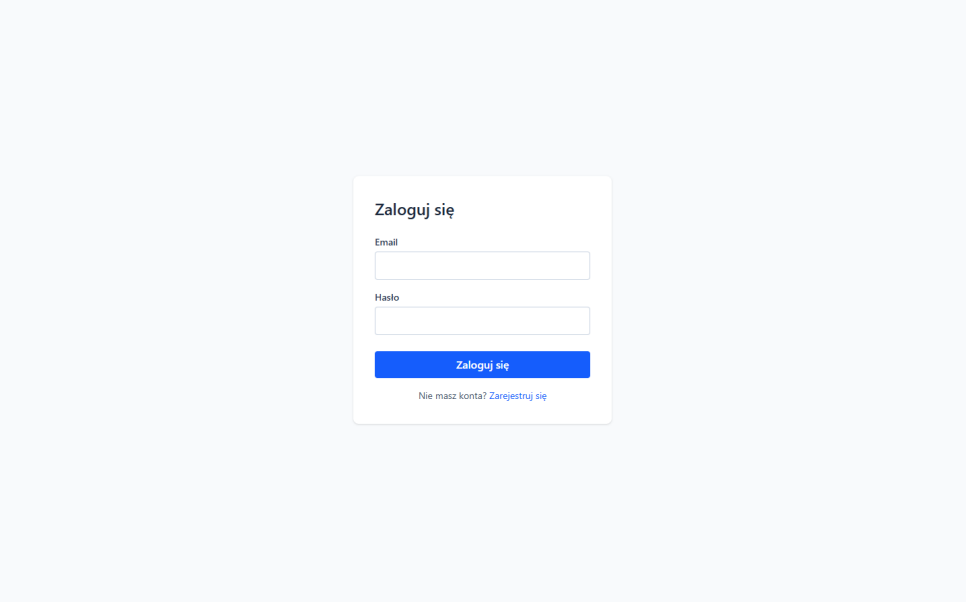
## 5.1 Frontend

Frontend to aplikacja jednostronicowa napisana w Blazor WebAssembly [4]. Wybrałem Blazora, bo pozwala pisać interfejs w C#, tym samym języku co backend, i korzystać z tych samych klas DTO. Wygląd oparłem na Tailwind CSS [15]. Zdefiniowałem kilka własnych klas, np. `btn-primary` i `data-table`, żeby wszystkie strony wyglądały tak samo.

Kod frontendu jest podzielony na kilka folderów:

- `Pages/` zawiera strony aplikacji, czyli logowanie i rejestrację, listy i formularze dla ośrodków, wydziałów, kierunków, przedmiotów, grup i sylabusów oraz siatkę przedmiotów,
- w `Services/ApiClients/` są klienci REST API, po jednym na każdy zasób,
- w `Auth/` jest `TokenAuthenticationStateProvider`, który odczytuje dane i role użytkownika, oraz `AuthorizationMessageHandler`, który dokleja token do zapytań,
- `Layout/` to układ strony z menu po lewej i informacją o zalogowanym użytkowniku.

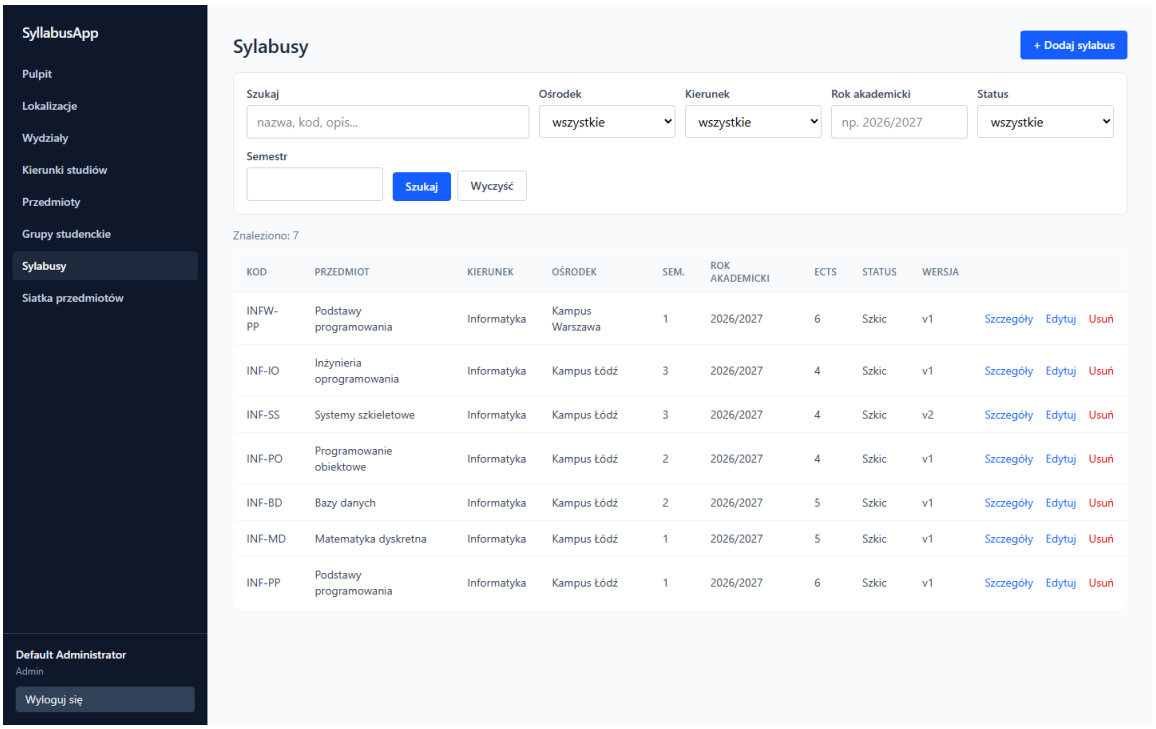
To, co widzi użytkownik, zależy od jego roli. Przyciski dodawania, edycji i usuwania pokazują się tylko administratorom i wykładowcom, czego pilnuje komponent `AuthorizeView`. Strony, które wymagają logowania, mają atrybut `[Authorize]`. Ekran logowania widać na rysunku 3.



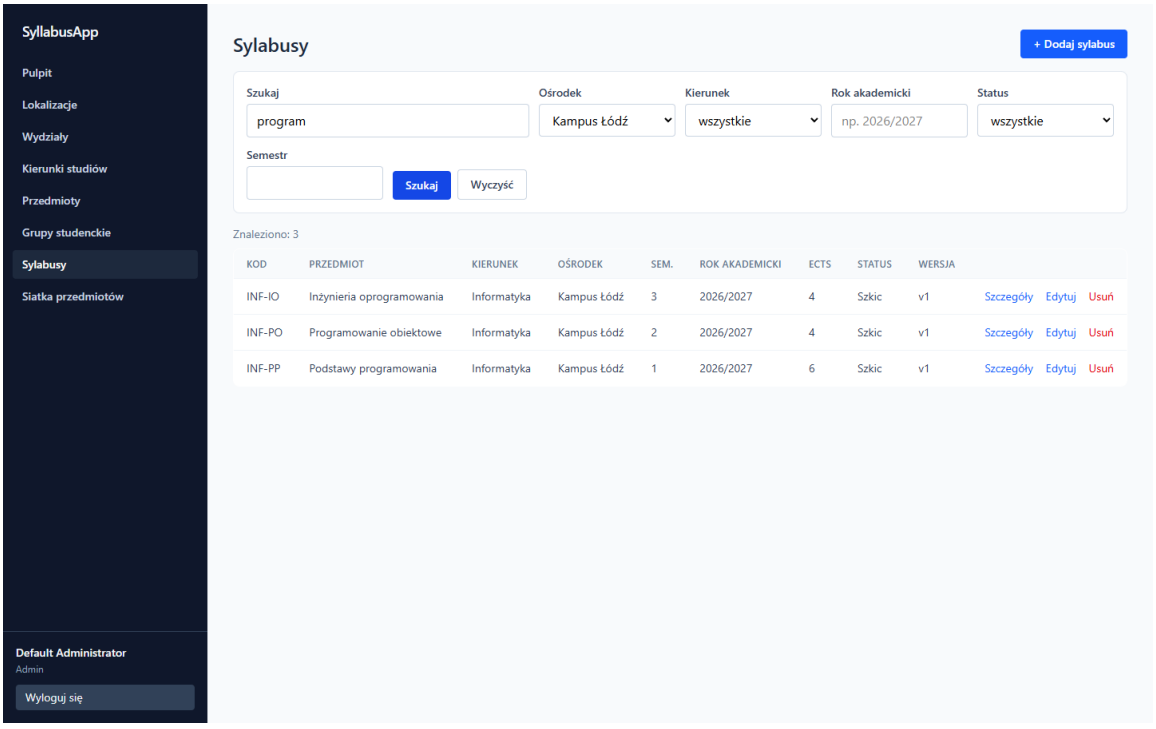

Rysunek 3: Ekran logowania

Nad listą sylabusów (rys. 4) jest panel wyszukiwania. Można wpisać fragment nazwy lub kodu przedmiotu, opisu albo celów, a wyniki zawęzić do ośrodka, kierunku, roku akademickiego,

statusu i semestru. Kryteria trafiają do API jako parametry zapytania, a filtrowanie dzieje się już w bazie danych (rys. 5).



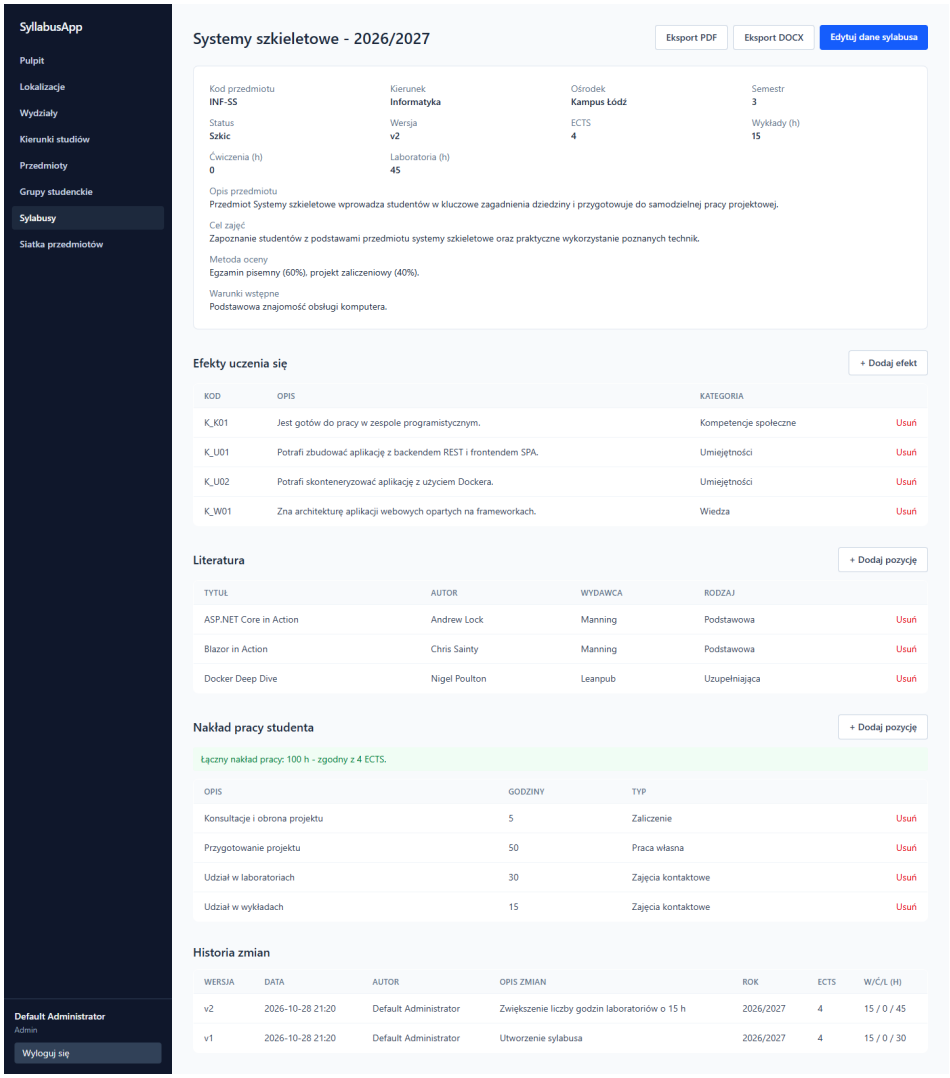

Rysunek 4: Lista sylabusów




Rysunek 5: Wyszukiwanie sylabusów po tekście „program” w ośrodku Kampus Łódź

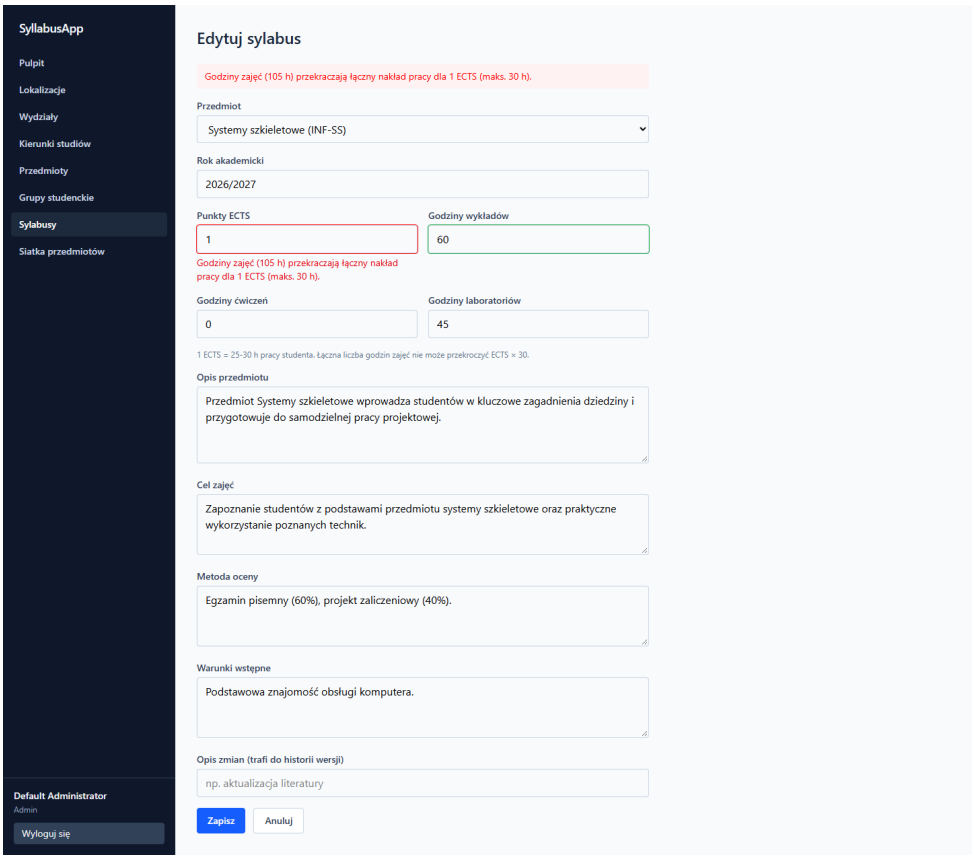
Na stronie szczegółów (rys. 6) widać wszystkie dane sylabusa: informacje podstawowe, opisy, efekty uczenia się, literaturę, nakład pracy studenta i historię zmian. Pod nakładem pracy

aplikacja od razu pokazuje, czy suma godzin pasuje do ECTS. Stąd też można pobrać sylabus jako PDF lub DOCX. Plik pobiera `HttpClient`, bo tylko on wysyła token, a na dysk zapisuje go krótka funkcja w JavaScripcie wywoływana przez `IJSRuntime`.



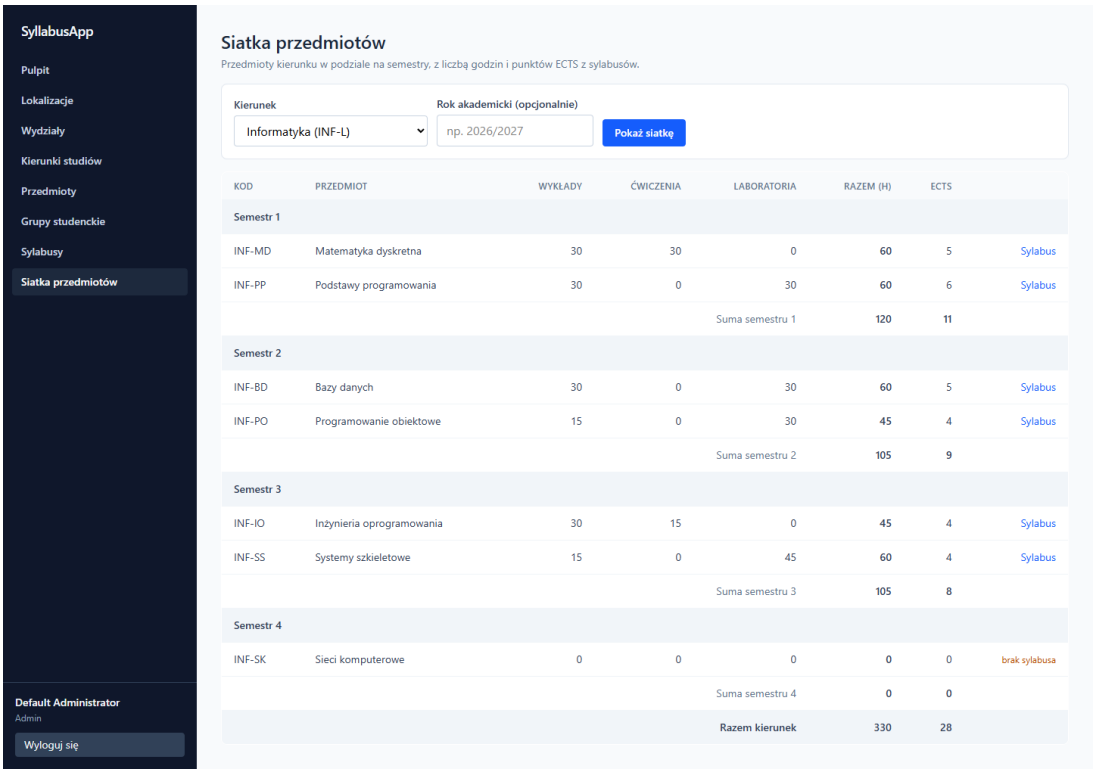

Rysunek 6: Szczegóły sylabusa z nakładem pracy i historią zmian

Formularz sylabusa (rys. 7) używa komponentu `EditForm` i walidacji opartej na atrybutach `DataAnnotations`. DTO formularza implementuje `IValidatableObject`, więc błędy w godzinach i punktach ECTS widać jeszcze przed wysłaniem danych na serwer. Przy edycji można wpisać krótki opis zmian, który trafia potem do historii wersji.




Rysunek 7: Walidacja liczby godzin w formularzu sylabusa

Siatka przedmiotów (rys. 8) pokazuje przedmioty wybranego kierunku pogrupowane w semestry. Przy każdym przedmiocie są godziny wykładów, ćwiczeń i laboratoriów oraz punkty ECTS, a na dole sumy dla semestru i całego kierunku. Jeśli przedmiot nie ma jeszcze sylabusa, widać to od razu.




Rysunek 8: Siatka przedmiotów kierunku Informatyka

## 5.2 Backend

Backend to aplikacja ASP.NET Core Web API [3]. Wszystkie zasoby obsługuję według jednego schematu. Kontroler przyjmuje i zwraca DTO, a samą pracę zleca serwisowi. Interfejs serwisu leży w `Application`, a implementacja w `Infrastructure` i to ona korzysta z `SyllabusDbContext`. Serwisy są rejestrowane w kontenerze zależności jako *scoped*. Kiedy operacja może się nie udać, bo np. nie ma takiego przedmiotu, serwis nie rzuca wyjątku, tylko zwraca obiekt wyniku, np. `SyllabusResult`. Kontroler zamienia go na właściwy kod HTTP: 200, 201, 400 albo 404.

### 5.2.1 Uwierzytelnianie i autoryzacja

Użytkowników i role obsługuje ASP.NET Core Identity [21]. Hasło musi mieć co najmniej 8 znaków, w tym cyfrę, małą i wielką literę. Po pięciu nieudanych próbach konto blokuje się na 15 minut. Gdy logowanie się powiedzie, `TokenService` tworzy token JWT podpisany algorytmem HMAC-SHA256. W tokenie są identyfikator, e-mail, imię, nazwisko i role użytkownika. Nowe konto dostaje rolę *Student*. Role i konto administratora tworzy przy starcie aplikacji klasa `DataSeeder`. Żeby zmieniać dane, trzeba mieć rolę *Admin* albo *Lecturer*, co pilnuje atrybut `[Authorize(Roles = ...)]`.

### 5.2.2 Research: logowanie kontem uczelnianym

Przy wymaganiu logowania kontem uczelnianym trzeba było najpierw sprawdzić, jakie są możliwości. Wziąłem pod uwagę dwie.

Pierwsza to OpenID Connect z Microsoft Entra ID [22, 23]. Większość polskich uczelni, w tym SAN, daje studentom i pracownikom konta Microsoft 365 w swojej domenie. Aplikację rejestruje się w Entra ID, a logowanie przebiega według schematu *Authorization Code* z PKCE. We frontendzie można użyć biblioteki MSAL dla Blazor WebAssembly, a backend sprawdza token wydany przez Entra ID. Żeby wpuszczać tylko osoby z jednej uczelni, ustawia się identyfikator dzierżawy, czyli *tenant ID*.

Druga możliwość to USOS API [10]. Uczelnie, które korzystają z USOS-a, udostępniają API z autoryzacją OAuth 1.0a. Można przez nie potwierdzić tożsamość studenta i pobrać np. jego numer albumu i kierunek. Trzeba jednak dostać klucz aplikacji od administratora USOS-a na danej uczelni.

Lepszym wyborem jest moim zdaniem Entra ID. Opiera się na otwartych standardach [24, 22], .NET dobrze go obsługuje i działa także na uczelniach bez USOS-a. W obecnej architekturze wystarczyłoby dodać w API drugi sposób uwierzytelniania i endpoint, który po sprawdzeniu tokenu z Entra ID znajdzie albo utworzy użytkownika po adresie e-mail i wyda mu zwykły token JWT aplikacji. Role, autoryzacja i frontend zostałyby bez zmian. Tej integracji nie uruchomiłem, bo rejestracja aplikacji w dzierżawie uczelni wymaga uprawnień administratora IT.

### 5.2.3 Wersjonowanie i historia zmian

Za każdym razem, gdy sylabus jest zapisywany, powstaje nowy rekord `SyllabusVersion`. Przy tworzeniu sylabusa jest to wersja 1 z opisem „Utworzenie sylabusa”, a każda kolejna edycja dodaje wersję z numerem o jeden większym. W wersji zapisuję najważniejsze pola sylabusa, autora, datę, opis zmian i pełną kopię sylabusa w JSON (listing 1). Unikalny indeks na identyfikatorze sylabusa i numerze wersji nie pozwala, żeby dwie wersje miały ten sam numer.

Listing 1: Zapis nowej wersji przy edycji sylabusa (fragment `SyllabusService`)

```
syllabus.ApplyUpdate(dto);
syllabus.UpdatedAt = DateTime.UtcNow;
await _dbContext.SaveChangesAsync();

var lastVersion = await _dbContext.SyllabusVersions
    .Where(v => v.SyllabusId == id)
    .MaxAsync(v => (int?)v.VersionNumber) ?? 0;
var description = string.IsNullOrWhiteSpace(dto.ChangeDescription)
    ? "Aktualizacja sylabusa" : dto.ChangeDescription.Trim();
await AddVersionAsync(syllabus, lastVersion + 1, description, userId);
```

### 5.2.4 Walidacja danych

Dane są sprawdzane na dwa sposoby. Atrybuty `DataAnnotations` pilnują pól wymaganych, długości tekstu i formatu roku akademickiego, np. 2026/2027. Reguły dotyczące godzin wydzieliłem do klasy `SyllabusRules` (listing 2). Zgodnie z zasadami ECTS [2] przyjąłem, że jeden punkt to 25–30 godzin pracy studenta. Z tego wynikają następujące reguły:

- ECTS musi być w zakresie od 1 do 30,
- godziny nie mogą być ujemne, a sylabus musi mieć przynajmniej jedną godzinę zajęć,
- wszystkie godziny zajęć razem nie mogą przekroczyć ECTS $\times$ 30,
- suma nakładu pracy studenta powinna wynosić od ECTS $\times$ 25 do ECTS $\times$ 30 godzin, a jeśli tak nie jest, na stronie sylabusa pojawia się ostrzeżenie.

Listing 2: Reguły walidacji godzin (fragment `SyllabusRules`)

```
public static List<string> ValidateHours(int ects, int lectureHours,
    int exerciseHours, int laboratoryHours)
{
    var errors = new List<string>();
    if (ects < MinEcts || ects > MaxEcts)
        errors.Add($"Liczba punktów ECTS musi mieścić się w zakresie {MinEcts}-{
            MaxEcts}.");
    if (lectureHours < 0 || exerciseHours < 0 || laboratoryHours < 0)
        errors.Add("Liczba godzin nie może być ujemna.");
    var contactHours = lectureHours + exerciseHours + laboratoryHours;
    if (contactHours <= 0)
        errors.Add("Sylabus musi mieć co najmniej jedną godzinę zajęć.");
    if (ects >= MinEcts && contactHours > ects * MaxHoursPerEcts)
        errors.Add($"Godziny zajęć ({contactHours} h) przekraczają ...");
    return errors;
}
```

Te reguły wywołuje klasa bazowa formularza `SyllabusEditDto`, która implementuje `IValidatableObject`. Projekt `Application` jest wspólny, więc ta sama walidacja działa w Blazorze, gdzie komunikat pojawia się pod polem, i w API, które zwraca wtedy kod 400.

### 5.2.5 Wyszukiwanie i siatka przedmiotów

Metoda `GetAllAsync(SyllabusFilterDto)` buduje zapytanie LINQ krok po kroku. Każde wypełnione pole filtra dokłada kolejny warunek `Where`. EF Core zamienia to na jedno zapytanie SQL ze złączeniami od sylabusa przez przedmiot, kierunek i wydział aż do ośrodka. Przy wyszukiwaniu tekstu wielkość liter nie ma znaczenia.

`GetCurriculumGridAsync` pobiera wszystkie przedmioty kierunku razem z ich sylabusami i grupuje je po semestrach. Dla każdego przedmiotu bierze sylabus z wybranego roku akademickiego, a jeśli rok nie jest podany, to najnowszy. Sumy godzin i ECTS liczą się same we właściwościach DTO.

### 5.2.6 Eksport do PDF i DOCX

Eksportem zajmuje się `SyllabusExportService`. Najpierw pobiera sylabus ze wszystkim, co z nim związane: przedmiotem, kierunkiem, wydziałem, ośrodkiem, prowadzącym, efektami, literaturą i nakładem pracy. Potem przygotowuje treść, która jest taka sama dla obu formatów: tabelę z danymi podstawowymi, sekcje z opisami i tabele efektów, literatury i nakładu pracy. PDF tworzy biblioteka QuestPDF [17] na licencji Community, w której układ strony opisuje się w kodzie C#. DOCX tworzy Open XML SDK [18], więc na serwerze nie trzeba mieć zainstalowanego Office'a. Polskie znaki wyświetlają się poprawnie w obu formatach.

## 5.3 Baza danych

Na bazę danych wybrałem PostgreSQL 16 [5]. Jest darmowy, sprawdzony i ma oficjalny obraz Dockera. Aplikacja łączy się z nim przez Entity Framework Core 10 [6] i sterownik Npgsql [16]. Pracowałem w podejściu *code first*: schemat bazy wynika z klas encji i konfiguracji w `SyllabusDbContext`, a każda zmiana schematu to nowa migracja. API przy starcie samo stosuje brakujące migracje (`Database.MigrateAsync()`), więc na nowej instalacji nie trzeba ręcznie zakładać tabel.

Tabela 6: Najważniejsze encje modelu danych

| **Encja** | **Opis i powiązania** |
|---|---|
| `Location` | Ośrodek akademicki (nazwa, adres, miasto); ma wiele wydziałów |
| `Faculty` | Wydział w ośrodku; opcjonalnie dziekan (`User`) |
| `FieldOfStudy` | Kierunek studiów (poziom, liczba semestrów) na wydziale |
| `Course` | Przedmiot kierunku z numerem semestru |
| `Syllabus` | Sylabus przedmiotu dla roku akademickiego: ECTS, godziny, opis, cele, ocenianie, wymagania, status, prowadzący |
| `LearningOutcome` | Efekt uczenia się (kod, opis, kategoria: wiedza, umiejętności, kompetencje społeczne) |
| `Literature` | Pozycja literatury (podstawowa lub uzupełniająca) |
| `WorkloadItem` | Pozycja nakładu pracy studenta (zajęcia kontaktowe, praca własna, zaliczenie) |
| `SyllabusVersion` | Wersja sylabusa: numer, opis zmian, autor, kopia pól i kopia JSON |
| `SyllabusApproval` | Wpis obiegu akceptacji (na razie nieużywany, przygotowany na przyszłość) |
| `StudentGroup` | Grupa studencka na kierunku |
| `User`, `Role` | Użytkownicy i role (rozszerzenie tabel ASP.NET Core Identity) |

Dzięki układowi `Location` → `Faculty` → `FieldOfStudy` → `Course` → `Syllabus` ten sam przedmiot może mieć osobne sylabusy w różnych ośrodkach. Pole `AcademicYear` pozwala dodatkowo trzymać osobne sylabusy dla kolejnych roczników. W konfiguracji modelu ustawiłem między innymi unikalne kody wydziałów i kierunków oraz kaskadowe usuwanie wersji razem z sylabusem. Zablokowałem też usunięcie użytkownika, który jest prowadzącym

albo koordynatorem jakiegoś sylabusa.

## 5.4 Docker

Każda część systemu ma własny obraz. Obraz API budowany jest w dwóch etapach. Najpierw `dotnet/sdk:10.0` kompiluje i publikuje aplikację, a potem gotowe pliki trafiają do dużo mniejszego obrazu `dotnet/aspnet:10.0`. Przy budowaniu frontendu najpierw powstaje arkusz Tailwind CSS, potem publikowana jest aplikacja Blazor, a gotowe pliki serwuje `nginx:alpine`. W konfiguracji nginx jest reguła `try_files`, która każdy nieznany adres kieruje do `index.html`. Bez niej odświeżenie np. strony `/syllabuses` kończyłoby się błędem 404. Baza korzysta z oficjalnego obrazu `postgres:16-alpine`, a dane zapisuje w wolumenie.

Wszystkie kontenery opisuje plik `docker-compose.yml` (listing 3). Ustawienia API, czyli połączenie z bazą, klucz JWT, konto administratora i dozwolone adresy CORS, przekazuję przez zmienne środowiskowe. ASP.NET Core traktuje je jako ważniejsze od wartości z `appsettings.json`. API rusza dopiero wtedy, gdy baza zgłosi, że jest gotowa.

Listing 3: Fragment pliku `docker-compose.yml`

```
  api:
    build:
      context: .
      dockerfile: src/SyllabusApp.Api/Dockerfile
    depends_on:
      db:
        condition: service_healthy
    environment:
      ConnectionStrings__DefaultConnection: "Host=db;Port=5432;..."
      Jwt__Key: "..."
      Cors__Origins__0: "http://localhost:8080"
    ports:
      - "8081:8080"
    healthcheck:
      test: ["CMD-SHELL", "curl -fs http://localhost:8080/health || exit 1"]
      interval: 15s
```

Żeby uruchomić cały system, wystarczy wpisać `docker compose up -d -build`. Frontend działa wtedy pod adresem `localhost:8080`, a API pod `localhost:8081`.

## 5.5 Testy

Testy napisałem w xUnit [19]. Serwisy testuję na bazie EF Core InMemory. Każdy test dostaje nową, pustą bazę w pamięci o losowej nazwie, więc testy nie wpływają na siebie nawzajem i nie potrzebują działającego PostgreSQL-a. W projekcie są 22 testy w trzech klasach (tab. 7).

Tabela 7: Zakres testów jednostkowych

| Klasa testowa | Liczba | Sprawdzane zachowanie |
|---|---|---|
| `SyllabusRulesTests` | 10 | Zakres ECTS, ujemne i zerowe godziny, przekroczenie limitu godzin, zgodność nakładu pracy z ECTS (wartości graniczne) |
| `SyllabusServiceTests` | 6 | Utworzenie wersji 1, dodanie wersji 2 z zachowaniem starych wartości, błąd dla nieistniejącego przedmiotu, filtrowanie po ośrodku, roku i tekście, siatka przedmiotów, eksport PDF i DOCX |
| `LocationServiceTests` | 6 | Dodawanie, edycja i usuwanie ośrodków, sortowanie, obsługa nieistniejących rekordów |

Na listingu 4 jest przykładowy test wersjonowania. Tworzy on sylabus z 30 godzinami laboratoriów, zmienia je na 45 i sprawdza, czy pojawiła się wersja 2, a wersja 1 nadal pamięta starą wartość. Testy uruchamia się poleceniem `dotnet test` i wszystkie przechodzą.

Listing 4: Test wersjonowania sylabusa

```
[Fact]
public async Task UpdateAsync_AddsNewVersion_AndKeepsOldValuesInHistory()
{
    using var db = CreateDb();
    var service = new SyllabusService(db);
    var dto = NewSyllabus(1);
    var created = (await service.CreateAsync(dto, LecturerId)).Data!;

    var update = ToUpdate(dto, "Więcej godzin laboratoriów");
    update.LaboratoryHours = 45;
    var result = await service.UpdateAsync(created.Id, update, LecturerId);

    Assert.Equal(2, result.Data!.CurrentVersion);
    var versions = (await service.GetVersionsAsync(created.Id))!.ToList();
    Assert.Equal([2, 1], versions.Select(v => v.VersionNumber));
    Assert.Equal(45, versions[0].LaboratoryHours);
    Assert.Equal(30, versions[1].LaboratoryHours);
}
```

## 5.6 Monitoring

Do monitorowania użyłem mechanizmu *health checks*, który jest wbudowany w ASP.NET Core [20]. W API dodałem sprawdzenie `AddDbContextCheck` dla `SyllabusDbContext`. Przy każdym wejściu na `/health` aplikacja próbuje połączyć się z bazą i odpowiada `Healthy` z kodem 200 albo `Unhealthy` z kodem 503.

Listing 5: Konfiguracja health checks w `Program.cs`

```
builder.Services.AddHealthChecks()
    .AddDbContextCheck<SyllabusDbContext>("database");
app.MapHealthChecks("/health");
```

Z tego adresu korzysta Docker. Co 15 sekund sprawdza `/health` kontenera `api`, a bazę sprawdza poleceniem `pg_isready`. Polecenie `docker compose ps` pokazuje przy każdej usłudze `healthy` albo `unhealthy`. Jeśli któryś kontener padnie, Docker sam go uruchomi ponownie. Do tego aplikacja zapisuje logi, m.in. zapytania SQL, błędy i zakładanie kont. Można je podejrzeć przez `docker compose logs api`.

# 6 Podsumowanie i wnioski

Aplikacja działa i spełnia wymagania projektu poza logowaniem kontem uczelnianym. Tego nie wdrożyłem, ale opisałem w researchu, jak można je dodać przez Microsoft Entra ID.

## 6.1 Napotkane wyzwania

Pierwszym problemem było to, żeby nie pisać tych samych reguł walidacji dwa razy. Rozwiązałem go wspólną klasą bazową formularza z `IValidatableObject`, która leży w projekcie używanym przez API i przez Blazora.

Drugi problem dotyczył enumeracji. .NET domyślnie wysyła je w JSON jako liczby, więc musiałem tak samo skonfigurować `JsonStringEnumConverter` po stronie API i frontendu. Inaczej deserializacja się wysypywała.

Kłopot sprawiło też pobieranie plików. Zwykły link nie wysyła tokenu JWT, więc API odrzucało takie zapytanie. Plik pobiera więc `HttpClient`, a zapisuje go na dysku krótka funkcja w JavaScripcie.

Przy budowaniu obrazu frontendu okazało się, że brakuje arkusza Tailwind CSS, bo nie jest on trzymany w repozytorium. Generuję go teraz w trakcie budowania obrazu, a adres API wpisuję do `appsettings.json` jako argument budowania. Ostatnia rzecz to pierwsze uruchomienie na pustej bazie w kontenerze. Żeby nie zakładać tabel ręcznie, API samo stosuje migracje przy starcie.

## 6.2 Możliwości dalszego rozwoju

- wdrożyć logowanie kontem uczelnianym przez Microsoft Entra ID, tak jak opisałem w researchu,
- dodać obieg akceptacji sylabusa przez dziekana, do którego jest już przygotowana encja `SyllabusApproval` i statusy,
- umożliwić porównanie dwóch wersji sylabusa i przywrócenie starszej,
- zrobić publiczny katalog opublikowanych sylabusów, dostępny bez logowania,
- rozbudować monitoring o metryki w Prometheusie i Grafanie oraz zbieranie logów w jednym miejscu,
- dopisać testy integracyjne API i testy interfejsu,
- powiązać efekty uczenia się przedmiotu z efektami kierunkowymi i Polską Ramą Kwalifikacji.

# Spis tabel


1 Wymagania funkcjonalne i sposób ich realizacji . . . . . 4
2 Porównanie istniejących rozwiązań z SyllabusApp . . . . . 6
3 Projekty (warstwy) rozwiązania . . . . . 8
4 Najważniejsze endpointy REST API . . . . . 9
5 Stos technologiczny . . . . . 10
6 Najważniejsze encje modelu danych . . . . . 18
7 Zakres testów jednostkowych . . . . . 20

# Spis rysunków

1 Architektura systemu SyllabusApp . . . . . 7
2 Interaktywna dokumentacja API w Scalarze . . . . . 9
3 Ekran logowania . . . . . 11
4 Lista sylabusów . . . . . 12
5 Wyszukiwanie sylabusów po tekście „program” w ośrodku Kampus Łódź . . . 12
6 Szczegóły sylabusa z nakładem pracy i historią zmian . . . . . 13
7 Walidacja liczby godzin w formularzu sylabusa . . . . . 14
8 Siatka przedmiotów kierunku Informatyka . . . . . 15

# Spis listingów

1 Zapis nowej wersji przy edycji sylabusa (fragment `SyllabusService`) . . . . 16
2 Reguły walidacji godzin (fragment `SyllabusRules`) . . . . . 17
3 Fragment pliku `docker-compose.yml` . . . . . 19
4 Test wersjonowania sylabusa . . . . . 20
5 Konfiguracja health checks w `Program.cs` . . . . . 20

# Bibliografia


[1] Sejm RP. *Ustawa z dnia 20 lipca 2018 r. – Prawo o szkolnictwie wyższym i nauce (Dz.U. 2018 poz. 1668)*. 2018. URL: https://isap.sejm.gov.pl/isap.nsf/DocDetails.xsp?id=WDU20180001668 (dostęp 28.10.2026).

[2] European Commission. *European Credit Transfer and Accumulation System (ECTS)*. URL: https://education.ec.europa.eu/education-levels/higher-education/inclusive-and-connected-higher-education/european-credit-transfer-and-accumulation-system (dostęp 28.10.2026).

[3] Microsoft. *ASP.NET Core documentation*. URL: https://learn.microsoft.com/aspnet/core/ (dostęp 28.10.2026).

[4] Microsoft. *ASP.NET Core Blazor*. URL: https://learn.microsoft.com/aspnet/core/blazor/ (dostęp 28.10.2026).

[5] The PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. URL: https://www.postgresql.org/docs/16/ (dostęp 28.10.2026).

[6] Microsoft. *Entity Framework Core*. URL: https://learn.microsoft.com/ef/core/ (dostęp 28.10.2026).

[7] Docker Inc. *Docker Docs – Docker Compose*. URL: https://docs.docker.com/compose/ (dostęp 28.10.2026).

[8] R. C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.

[9] Międzyuniwersyteckie Centrum Informatyzacji. *Uniwersytecki System Obsługi Studiów USOS*. URL: https://www.usos.edu.pl/ (dostęp 28.10.2026).

[10] Międzyuniwersyteckie Centrum Informatyzacji. *USOS API – dokumentacja*. URL: https://apps.usos.edu.pl/developers/api/ (dostęp 28.10.2026).

[11] Akademus. *Akademus – system wspomagający zarządzanie uczelnią*. URL: https://www.akademus.pl/ (dostęp 28.10.2026).

[12] Asseco Data Systems. *Uczelnia.XP – system zarządzania uczelnią*. URL: https://www.uczelnia.xp.pl/ (dostęp 28.10.2026).

[13] M. Jones, J. Bradley i N. Sakimura. *RFC 7519: JSON Web Token (JWT)*. 2015. URL: https://www.rfc-editor.org/rfc/rfc7519 (dostęp 28.10.2026).

[14] Scalar. *Scalar API Reference for .NET*. URL: https://github.com/scalar/scalar (dostęp 28.10.2026).

[15] Tailwind Labs. *Tailwind CSS Documentation*. URL: https://tailwindcss.com/docs (dostęp 28.10.2026).

[16] Npgsql Development Team. *Npgsql Entity Framework Core Provider*. URL: https://www.npgsql.org/efcore/ (dostęp 28.10.2026).

[17] QuestPDF. *QuestPDF – Modern PDF library for C# developers*. URL: https://www.questpdf.com/ (dostęp 28.10.2026).

[18] Microsoft. *Open XML SDK*. URL: https://learn.microsoft.com/office/open-xml/open-xml-sdk (dostęp 28.10.2026).

[19] .NET Foundation. *xUnit.net – Getting Started*. URL: https://xunit.net/ (dostęp 28.10.2026).

[20] Microsoft. *Health checks in ASP.NET Core*. URL: https://learn.microsoft.com/aspnet/core/host-and-deploy/health-checks (dostęp 28.10.2026).

[21] Microsoft. *Introduction to Identity on ASP.NET Core*. URL: https://learn.microsoft.com/aspnet/core/security/authentication/identity (dostęp 28.10.2026).

[22] OpenID Foundation. *OpenID Connect Core 1.0*. URL: https://openid.net/specs/openid-connect-core-1_0.html (dostęp 28.10.2026).

[23] Microsoft. *Microsoft identity platform and OpenID Connect protocol*. URL: https://learn.microsoft.com/entra/identity-platform/v2-protocols-oidc (dostęp 28.10.2026).

[24] D. Hardt. *RFC 6749: The OAuth 2.0 Authorization Framework*. 2012. URL: https://www.rfc-editor.org/rfc/rfc6749 (dostęp 28.10.2026).